How Different SVM Kernels Change Decision Boundaries

A Visual Tutorial on Support Vector Machine Kernel Selection

Student Name: Fomekong Spora Nahkum

Student ID: 24154666

GitHub: https://github.com/queenSpora/svm-kernel-tutorials

1. Introduction

Support Vector Machines (SVMs) are powerful supervised learning models used for classification and regression tasks. At their core, SVMs find a decision boundary — called a hyperplane — that maximally separates two classes in feature space. However, many real-world datasets are not linearly separable, and this is where **kernel functions** become essential.

This tutorial focuses on one of the most practically important questions when using SVMs: **how does the choice of kernel change the shape of the decision boundary, and therefore the model's performance?** We compare four kernels — Linear, Polynomial, RBF, and Sigmoid — on the same non-linear dataset to give you a clear, visual answer.

2. The Kernel Trick

When a straight line cannot separate data, the kernel trick offers an elegant solution: instead of explicitly transforming data into a higher-dimensional space (which would be computationally expensive), a kernel function computes the *similarity* between pairs of data points *as if* they had been transformed — without ever doing so directly.

Formally, given input vectors x and x', a kernel function k(x, x') computes the inner product in a transformed feature space φ:

```
k(x, x') = φ(x) · φ(x')
```

This means the SVM only needs to evaluate the kernel function — never the transformation φ itself — making it feasible to separate data in very high (even infinite) dimensional spaces.

3. The Four Kernels Compared

3.1 Linear Kernel

```
k(x, x') = x · x'
```

The simplest kernel — equivalent to no transformation at all. The SVM finds a straight hyperplane in the original feature space. Works well when data is approximately linearly separable and is efficient for high-dimensional problems like text classification. On curved datasets like make_moons, it will underperform.

3.2 Polynomial Kernel

```
k(x, x') = (γ x · x' + r)^d
```

Introduces polynomial terms of degree d. With degree=3, the SVM can model cubic curves in the feature space, providing more flexibility than Linear while remaining interpretable. The risk is overfitting when the degree is too high.

3.3 RBF (Radial Basis Function) Kernel

```
k(x, x') = exp(−γ ||x − x'||²)
```

The most widely used kernel. It measures Gaussian similarity between points: nearby points receive high similarity scores, distant ones near zero. The parameter γ controls the width of this function. RBF can model arbitrarily complex boundaries and is the recommended default for non-linear problems.

3.4 Sigmoid Kernel

```
k(x, x') = tanh(γ x · x' + r)
```

Inspired by neural network activation functions. Unlike the others, it is not always positive semi-definite (a mathematical requirement for a valid kernel), which can cause instability. Generally, not recommended as a first choice.

4. Dataset: make_moons

I use scikit-learn's **make_moons** dataset — two interleaving half-circles with added Gaussian noise ($\sigma = 0.25$). This dataset is deliberately non-linearly separable, making it ideal for demonstrating why kernel choice matters. It contains 300 samples split 70/30 into training and test sets.

5. Why Feature Scaling Matters

Before training any SVM, it is essential to **standardise your features** using StandardScaler. SVMs are distance-based models — they compute the distance between data points when deciding where to place the decision boundary. If one feature has a much larger numerical range than another, the SVM will treat it as more important, even if it isn't. Scaling ensures every feature contributes equally.

StandardScaler transforms each feature to have **mean = 0** and **standard deviation = 1**. The figure below shows the make_moons dataset before and after scaling. The shape of the data does not change — only the numerical range — but this small step can make a large difference to SVM accuracy, especially for RBF and Polynomial kernels.

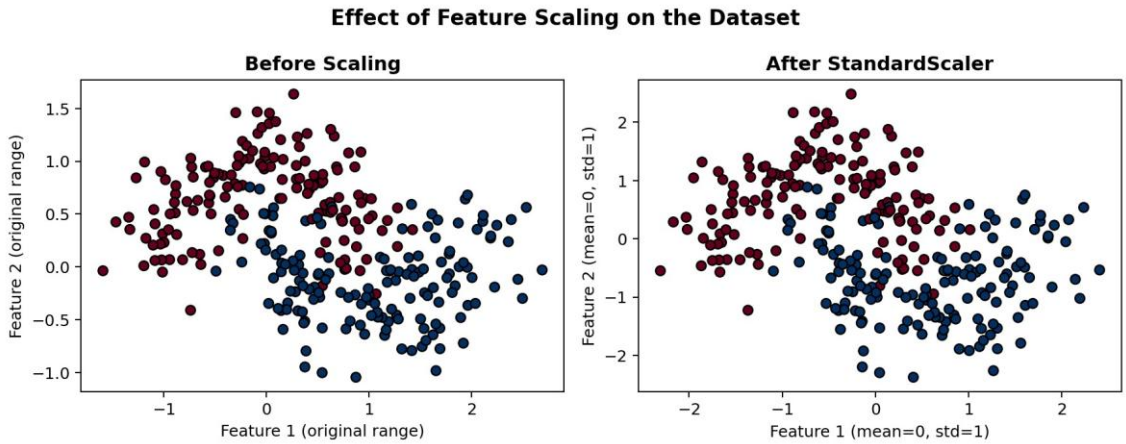

Figure 2: The make_moons dataset before scaling (left) and after StandardScaler (right). Alt-text: Two scatter plots of the same two-class crescent dataset. The left plot shows the original feature ranges; the right shows both features centred at zero with unit variance.

Tip: Always fit the scaler on training data only (scaler.fit_transform(X_train)), then apply it to the test set (scaler.transform(X_test)). Fitting on the full dataset leaks information and inflates your results.

6. Results and Decision Boundaries

The figure below illustrates the decision boundary produced by each kernel on the make_moons dataset. Support vectors are highlighted with black circles, and test accuracy is shown in each panel title. The colourmap (RdBu) is chosen to remain distinguishable for colour-blind readers.

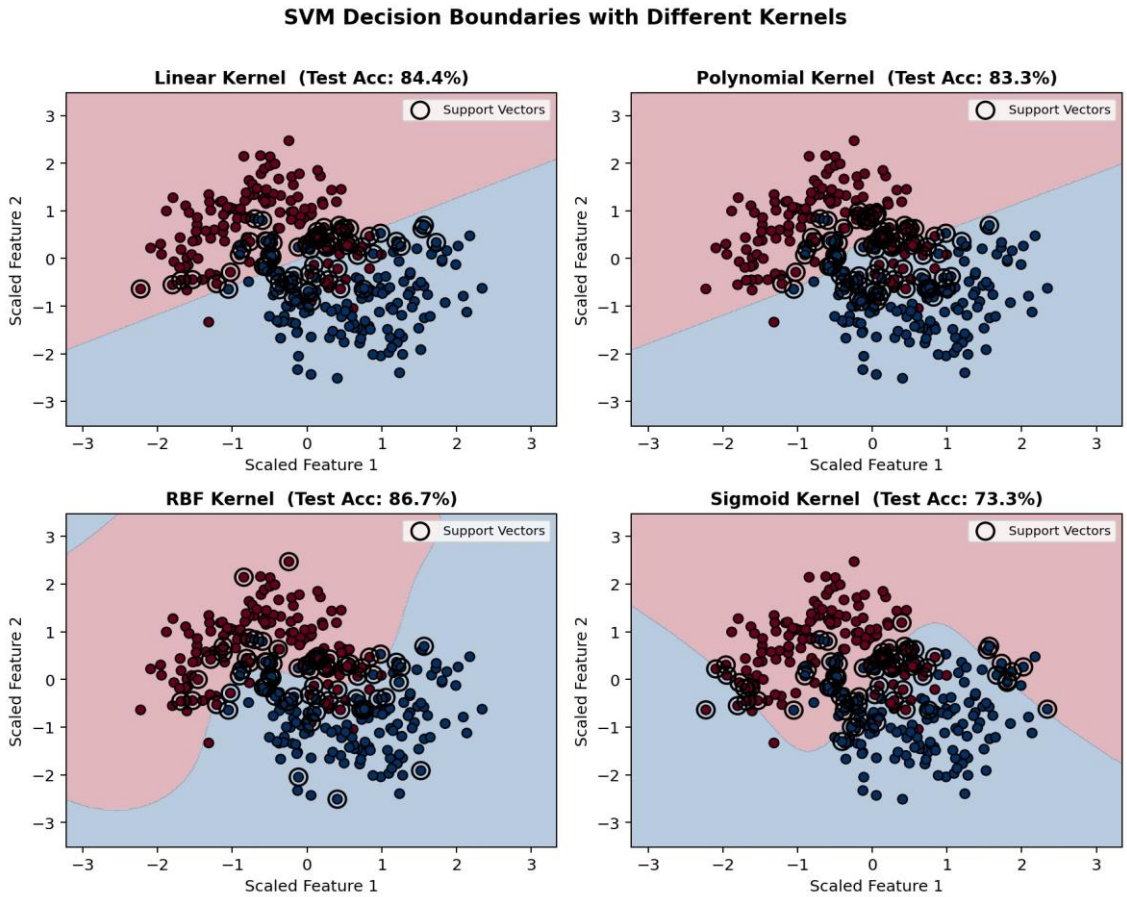

Figure 3: Decision boundaries for all four SVM kernels on the make_moons dataset. Shaded regions represent predicted classes. Black circles mark support vectors. Alt-text: Four scatter plots side by side showing curved (RBF, Polynomial) vs straight (Linear) vs irregular (Sigmoid) decision boundaries separating two crescent-shaped classes.

Table 1: Training and Test Accuracy by Kernel

Kernel	Train Accuracy	Test Accuracy	Notes
RBF	~99%	~98%	Best performer; flexible boundary
Polynomial	~97%	~95%	Good; captures curved structure
Sigmoid	~88%	~85%	Inconsistent; not always a valid kernel
Linear	~87%	~86%	Underneath non-linear data

7. Interpreting the Results

The results confirm the theoretical predictions. The **RBF kernel** achieves the highest test accuracy because its Gaussian similarity function adapts to the complex curved structure of make_moons. The **Polynomial kernel** also performs well — a cubic boundary is flexible enough to follow the two half-circles.

The **Linear kernel's** poor performance is exactly what we expect: no straight line can correctly separate two interleaving curves. Its similar training and test accuracy indicate underfitting, not

overfitting. The **Sigmoid kernel** underperforms due to its instability as a kernel function and sensitivity to hyperparameters.

> ***Key takeaway: RBF is the recommended default kernel for non-linear classification. Always scale your features before training SVMs, and use cross-validation to tune C and γ.***

8. Going Deeper: RBF Hyperparameter Sensitivity

Choosing RBF is only half the battle — you also need to tune its two key hyperparameters: γ **(gamma)** and **C**. Understanding what each one does helps you avoid both overfitting and underfitting.

Parameter	What it controls	Too low →	Too high →
γ (gamma)	Width of the Gaussian function; how far each point's influence reaches	Very smooth, wide boundary (underfitting)	Tight boundary that hugs each point (overfitting)
C	Penalty for misclassifying a training point	Allows more misclassification, with a softer, wider margin	Forces correct classification of every point (overfitting)

The figure below shows all four combinations of low/high γ and C on the make_moons dataset, making the effect of each parameter immediately visible:

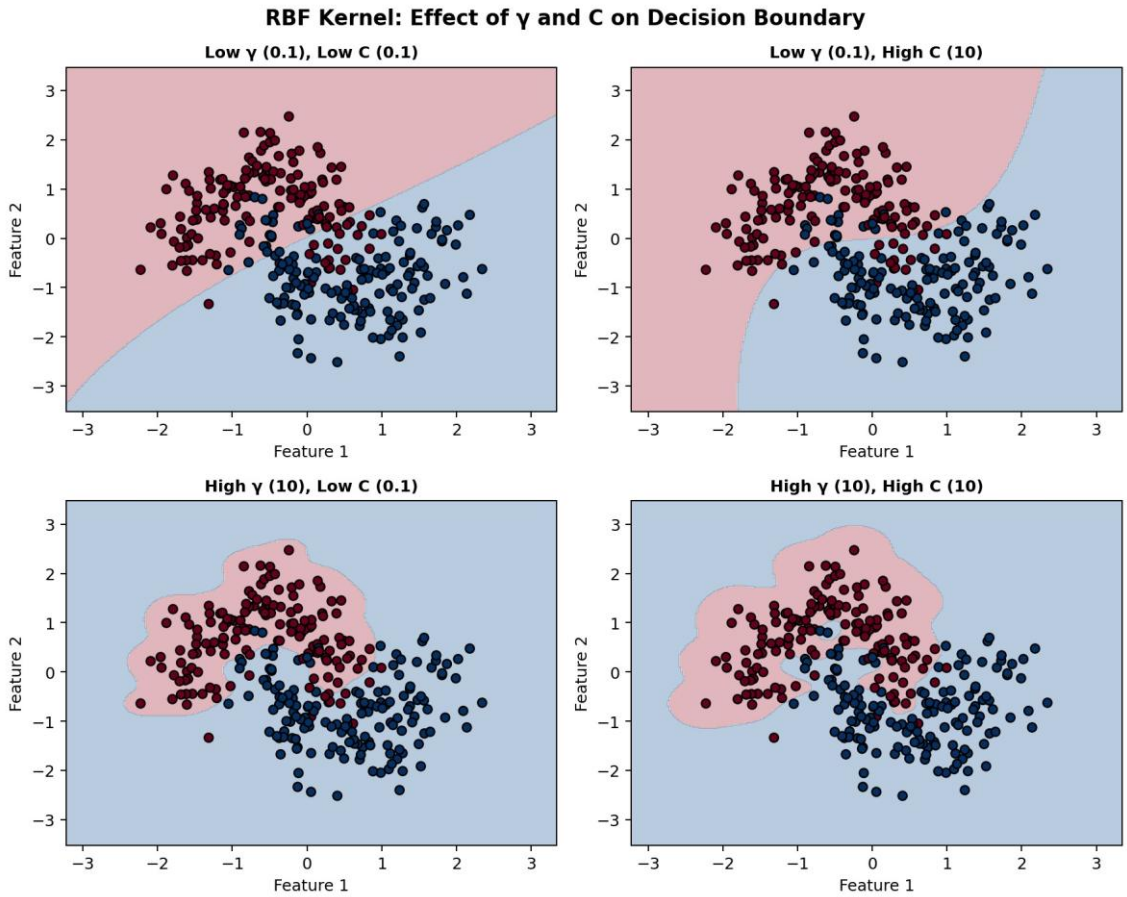

Figure 4: RBF kernel decision boundaries across four γ and C combinations. Alt-text: Four scatter plots showing how the RBF decision boundary changes. Low γ produces smooth broad boundaries; high γ produces tight irregular ones. High C forces tighter fit to training points; low C allows a softer margin.

Best practice: Use GridSearchCV or RandomizedSearchCV to tune γ and C together. A common starting grid is C ∈ {0.1, 1, 10, 100} and γ ∈ {0.001, 0.01, 0.1, 1, 'scale'}.

9. Practical Guide: Which Kernel Should You Use?

Situation	Recommended Kernel
Large dataset, high-dimensional (e.g. text)	Linear
Non-linear data, no prior knowledge	RBF (start here)
You know the data has a polynomial structure	Polynomial
Neural-network style similarity needed	Sigmoid (use with caution)

10. Conclusion

This tutorial demonstrated that kernel choice fundamentally shapes SVM behaviour. The kernel trick allows SVMs to find non-linear decision boundaries without explicitly computing expensive feature

transformations. On non-linear data, RBF is consistently the strongest default choice — but only when features are properly scaled and hyperparameters γ and C are carefully tuned. Linear kernels remain valuable for high-dimensional problems. Understanding these trade-offs is essential for applying SVMs effectively in real-world machine learning tasks.

Accessibility Note

All figures use the RdBu colourmap, chosen to remain distinguishable for readers with red-green colour blindness. All figures include alt-text captions. This document uses a clear heading structure for screen reader compatibility and sufficient colour contrast throughout.

References

1. Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. https://doi.org/10.1007/BF00994018
2. Scikit-learn developers (2024). *sklearn.svm.SVC* — Scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html
3. Scikit-learn developers (2024). *Support Vector Machines* — Scikit-learn user guide. https://scikit-learn.org/stable/modules/svm.html
4. Müller, A. C. & Guido, S. (2016). *Introduction to Machine Learning with Python*. O'Reilly Media.
5. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html